Brands
Vercel
Next.js
Turbo
v0
eve
AI SDK

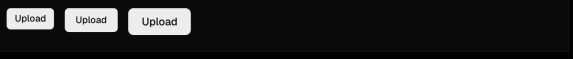
Components
Address Input
Avatar
Badge
Banner
Book
Breadcrumbs
Browser
Button
Calendar
Checkbox
Choicebox
Clearable Input
Code
Code Block
Collapse
Combobox
Command Menu
Context Card
Context Menu

Button

Trigger an action or event, such as submitting a form or displaying a dialog.

Sizes

The default size is medium.

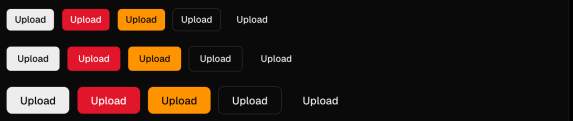


Hide code

```tsx
import { Button } from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <div className="flex flex-col md:flex-row items-start gap-4 flex-initial">
      <Button size="small">Upload</Button>
      <Button>Upload</Button>
      <Button size="large">Upload</Button>
    </div>
  );
}
```

All Types and Sizes in comparison



Hide code

```tsx
import { Button } from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <div className="flex flex-col gap-6">
      <div className="flex items-center gap-3">
        <Button size="small" variant="default">
          Upload
        </Button>
        <Button size="small" variant="error">
          Upload
        </Button>
        <Button size="small" variant="warning">
          Upload
        </Button>
        <Button size="small" variant="secondary">
          Upload
        </Button>
        <Button size="small" variant="tertiary">
          Upload
        </Button>
      </div>
      <div className="flex items-center gap-3">
        <Button variant="default">Upload</Button>
        <Button variant="error">Upload</Button>
        <Button variant="warning">Upload</Button>
        <Button variant="secondary">Upload</Button>
        <Button variant="tertiary">Upload</Button>
      </div>
      <div className="flex items-center gap-3">
        <Button size="large" variant="default">
          Upload
        </Button>
        <Button size="large" variant="error">
          Upload
        </Button>
        <Button size="large" variant="warning">
          Upload
        </Button>
        <Button size="large" variant="secondary">
          Upload
        </Button>
        <Button size="large" variant="tertiary">
          Upload
        </Button>
      </div>
    </div>
  );
}
```

Shapes

Icon-only buttons should include the `svgOnly` prop and an `aria-label`.



Hide code

```tsx
import { Button } from '@vercel/geistcn/components';
import type { JSX } from 'react';
import { IconArrowUp } from '@vercel/geistcn-assets/icons';

export function Component(): JSX.Element {
  return (
    <div className="flex flex-col md:flex-row items-start gap-4 flex-initial">
      <Button aria-label="Upload" shape="square" size="tiny" svgOnly>
        <IconArrowUp />
      </Button>
      <Button aria-label="Upload" shape="square" size="small" svgOnly>
        <IconArrowUp />
      </Button>
      <Button aria-label="Upload" shape="square" svgOnly>
        <IconArrowUp />
      </Button>
      <Button aria-label="Upload" shape="square" size="large" svgOnly>
        <IconArrowUp />
      </Button>
```

```tsx
      <Button aria-label="Upload" shape="circle" size="tiny" svgOnly>
        <IconArrowUp />
      </Button>
      <Button aria-label="Upload" shape="circle" size="small" svgOnly>
        <IconArrowUp />
      </Button>
      <Button aria-label="Upload" shape="circle" svgOnly>
        <IconArrowUp />
      </Button>
      <Button aria-label="Upload" shape="circle" size="large" svgOnly>
        <IconArrowUp />
      </Button>
    </div>
  );
}
```

Prefix and suffix



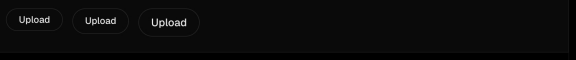
Hide code

```tsx
import { Button } from '@vercel/geistcn/components';
import type { JSX } from 'react';
import { IconArrowLeft, IconArrowRight } from '@vercel/geistcn-assets/icons';

export function Component(): JSX.Element {
  return (
    <div className="flex flex-col md:flex-row items-start gap-4 flex-initial">
      <Button prefix={<IconArrowLeft />}>Upload</Button>
      <Button suffix={<IconArrowRight />}>Upload</Button>
      <Button prefix={<IconArrowLeft />} suffix={<IconArrowRight />}>
        Upload
      </Button>
    </div>
  );
}
```

Rounded

Combination of `shape="rounded"` and the `shadow` prop, often used on marketing pages.

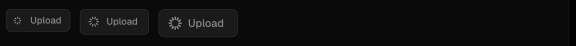


Hide code

```tsx
import { Button } from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <div className="flex flex-col md:flex-row items-start gap-4 flex-initial">
      <Button shadow shape="rounded" size="small" variant="secondary">
        Upload
      </Button>
      <Button shadow shape="rounded" variant="secondary">
        Upload
      </Button>
      <Button shadow shape="rounded" size="large" variant="secondary">
        Upload
      </Button>
    </div>
  );
}
```

Loading

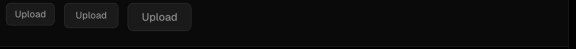


Hide code

```tsx
import { Button } from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <div className="flex flex-col md:flex-row items-start gap-4 flex-initial">
      <Button loading size="small">
        Upload
      </Button>
      <Button loading>Upload</Button>
      <Button loading size="large">
        Upload
      </Button>
    </div>
  );
}
```

Disabled

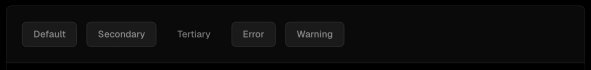


Hide code

```tsx
import { Button } from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <div className="flex flex-col md:flex-row items-start gap-4 flex-initial">
      <Button disabled size="small">
        Upload
      </Button>
      <Button disabled>Upload</Button>
      <Button disabled size="large">
        Upload
      </Button>
    </div>
  );
}
```

Disabled variants



Hide code

```tsx
import { Button } from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <div className="flex flex-col md:flex-row items-start gap-4 flex-initial">
      <Button disabled>Default</Button>
      <Button disabled variant="secondary">
        Secondary
      </Button>
      <Button disabled variant="tertiary">
        Tertiary
      </Button>
      <Button disabled variant="error">
        Error
      </Button>
      <Button disabled variant="warning">
        Warning
      </Button>
    </div>
  );
}
```

Link

Use `ButtonLink` for links with the same props as `Button`.



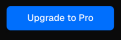
Hide code

```tsx
import { ButtonLink } from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <ButtonLink className="w-fit" href="#">
      Sign Up
    </ButtonLink>
  );
}
```

Custom

Use `CustomButton` to override colors for foreground, background, and border across normal, hover, and active states.



Hide code

```tsx
import { CustomButton } from '@vercel/geistcn/components';
import type { JSX } from 'react';

export function Component(): JSX.Element {
  return (
    <div className="flex flex-col md:flex-row items-start gap-4 flex-initial">
      <CustomButton
        active={{
          foreground: '#fff',
          background: 'var(--ds-blue-700)',
          border: 'var(--ds-blue-700)',
        }}
        hover={{
          foreground: '#fff',
          background: '#0B7BFE',
          border: 'var(--ds-blue-700)',
        }}
        normal={{
          foreground: '#fff',
          background: 'var(--ds-blue-700)',
          border: 'var(--ds-blue-700)',
        }}
        width={160}
      >
        Upgrade to Pro
      </CustomButton>
    </div>
  );
}
```

Best Practices

- Use `Button` for actions that mutate state (deploy, save, delete); use `ButtonLink` for navigation that changes the URL. Switch to a **Menu** or **Split Button** when more than one related action shares a row.
- Default `Button` is primary. Pass `type="secondary"` for the supporting action and `type="error"` for destructive confirmations. `primary`, `success`, `ghost`, and `violet` are not valid `type` values.
- For form submits, use `typeName="submit"`. The HTML `type` attribute lives on `typeName`, not on `type`, which controls the visual variant.
- Pass `loading` instead of swapping in a spinner so the button stays focusable and announces the busy state to assistive tech.
- Disable a button only when the action is impossible right now (missing input, insufficient permission); pair with a **Tooltip** that explains why.
- Title Case the label and name what happens: `Deploy Project`, `Invite Member`, `Rotate Key`. Avoid bare verbs (`Submit`) and generic confirms (`OK`, `Confirm`).
- Destructive buttons follow `Verb + Noun` and pair 1:1 with their toast: `Delete Project` then `Project deleted`. Mode-switch buttons append `Instead`: `Use a Recovery Code Instead`.
- Icon-only buttons require both `svgOnly` and `aria-label`; the validator throws without them. The `aria-label` names the action and the target (`Copy deployment URL`), not the icon (`Copy`).
- Don't set `aria-label` on a button that already has visible text; it overrides the label and creates a screen-reader mismatch.